



Assignment 3

Name: Matanat Heydarli 18080

Instructor: Nurlan Imanov

CRN: 20913

Submission date: 14.05.2026

1. Task 1

Apache Kafka Installation

I installed Kafka using the Mac OS installer, not using Docker. I first checked that I had Java installed; I had already installed Java (version 17). After that, I formatted the Kafka storage using KRaft mode. I used this command to start Kafka after formatting the storage area.

“bin/kafka-server-start.sh config/server.properties”

My terminal which showed that my Kafka server was successfully started:

“Kafka Server started”

This confirms that my Apache Kafka installation is running locally

SS1: Apache Kafka server started successfully

```
[2026-05-11 13:50:11,736] INFO [BrokerServer id=1] Transition from STARTING to STARTED (kafka.server.BrokerServer)
[2026-05-11 13:50:11,737] INFO Kafka version: 4.2.0 (org.apache.kafka.common.utils.AppInfoParser)
[2026-05-11 13:50:11,737] INFO Kafka commitId: a18251bae0b825c6 (org.apache.kafka.common.utils.AppInfoParser)
[2026-05-11 13:50:11,737] INFO Kafka startTimeMs: 1778493011736 (org.apache.kafka.common.utils.AppInfoParser)
[2026-05-11 13:50:11,737] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

Kafka Broker Verification

After starting Kafka, I tested the broker from another terminal using this command:

“bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092”

The output showed:

“localhost:9092 (id: 1 rack: null isFenced: false)”

This confirms that the Kafka broker was reachable on localhost:9092. The broker ID was 1.

SS 2: Kafka broker is reachable on localhost:9092

```
(base) metanet@Metanets-MacBook-Air ~ % cd ~/kafka-assignment/kafka_2.13-4.2.0
bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092
localhost:9092 (id: 1 rack: null isFenced: false) -> (
  Produce(0): 0 to 13 [usable: 13],
  Fetch(1): 4 to 18 [usable: 18],
  ListOffsets(2): 1 to 11 [usable: 11],
  Metadata(3): 0 to 13 [usable: 13],
  OffsetCommit(8): 2 to 10 [usable: 10],
  OffsetFetch(9): 1 to 10 [usable: 10],
  FindCoordinator(10): 0 to 6 [usable: 6],
  JoinGroup(11): 0 to 9 [usable: 9],
  Heartbeat(12): 0 to 4 [usable: 4],
  LeaveGroup(13): 0 to 5 [usable: 5],
  SyncGroup(14): 0 to 5 [usable: 5],
  DescribeGroups(15): 0 to 6 [usable: 6],
  ListGroups(16): 0 to 5 [usable: 5],
  SaslHandshake(17): 0 to 1 [usable: 1],
  ApiVersions(18): 0 to 4 [usable: 4],
  CreateTopics(19): 2 to 7 [usable: 7],
  DeleteTopics(20): 1 to 6 [usable: 6],
  DeleteRecords(21): 0 to 2 [usable: 2],
  InitProducerId(22): 0 to 5 [usable: 5],
  OffsetForLeaderEpoch(23): 2 to 4 [usable: 4],
  AddPartitionsToTxn(24): 0 to 5 [usable: 5],
  AddOffsetsToTxn(25): 0 to 4 [usable: 4],
  EndTxn(26): 0 to 5 [usable: 5],
  WriteTxnMarkers(27): 1 to 2 [usable: 2],
  TxnOffsetCommit(28): 0 to 5 [usable: 5],
  DescribeAcls(29): 1 to 3 [usable: 3],
  CreateAcls(30): 1 to 3 [usable: 3],
  DeleteAcls(31): 1 to 3 [usable: 3],
  DescribeConfigs(32): 1 to 4 [usable: 4],
  AlterConfigs(33): 0 to 2 [usable: 2],
  AlterReplicaLogDirs(34): 1 to 2 [usable: 2],
  DescribeLogDirs(35): 1 to 4 [usable: 4],
```

Kafka UI Installation and Startup

I completed the installation for my Kafka User Interface by downloading the Kafka UI Java Archive file (JAR). I created an application.yaml file in order to link the Kafka User Interface to my local Kafka broker.

My application.yaml file was configured as follows:

“server:

port: 8080

kafka:

clusters:

- name: local-kafka

bootstrapServers: localhost:9092”

I used the following command to start my Kafka User Interface:

“java -Dspring.config.additional-location=application.yml -jar kafka-ui-api-v0.7.2.jar”

The Kafka User Interface was successfully started and connected to my local Kafka broker on port 8080.

Kafka UI Browser Verification

I then accessed my Kafka User Interface through Safari by entering the following URL:

<http://localhost:8080>

The Kafka User Interface Dashboard indicated that the cluster with the name "local-kafka" was online.

The **Broker** section of the Kafka User Interface displayed my available broker along with the following information:

“Broker ID: 1

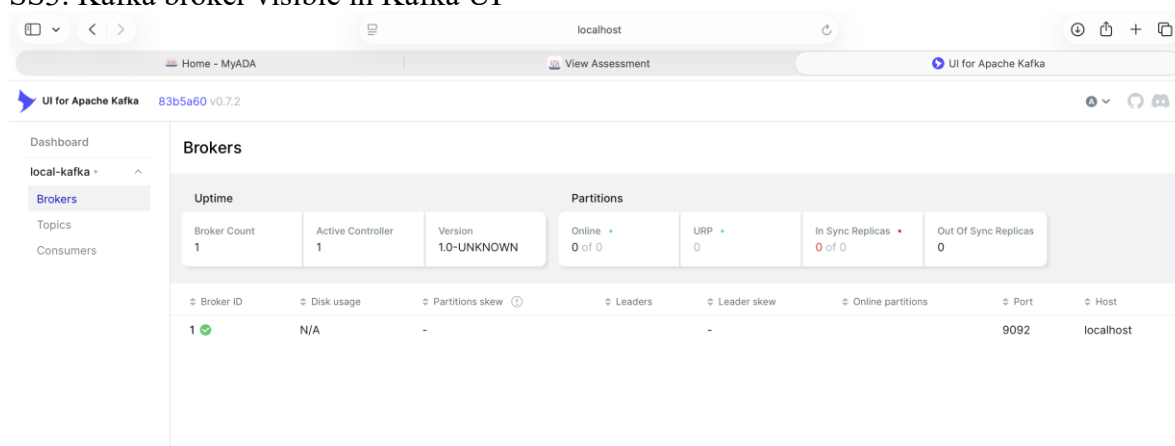
Host: localhost

Port: 9092

Broker Count: 1”

This indicates that the Kafka User Interface successfully connected to my local Kafka broker

SS3: Kafka broker visible in Kafka UI

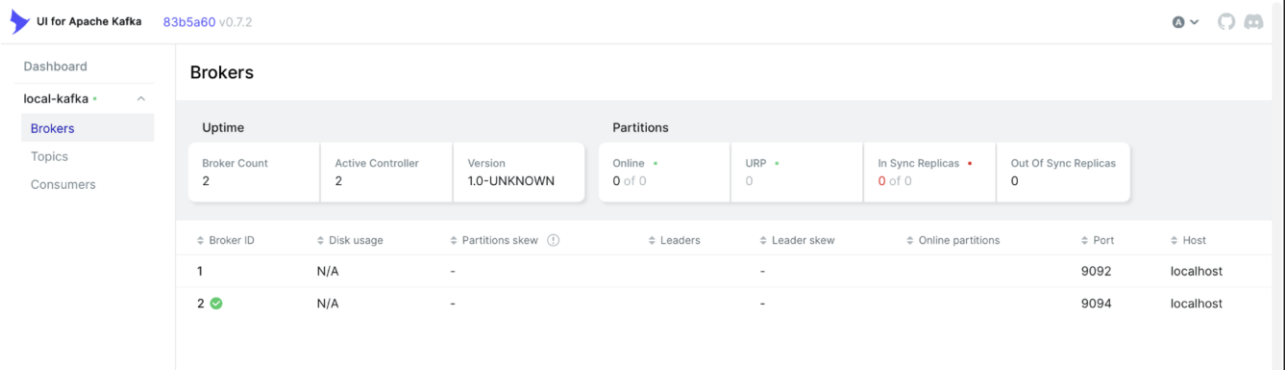


2. Task 2

Kafka Brokers Verification

Before creating my topic, I ensured that I had access to two Kafka brokers. To create a topic that has a replication factor of two, I need to have access to at least two Kafka brokers.

SS 4: Two Kafka brokers available in Kafka UI



The screenshot shows the 'Brokers' page in the Kafka UI. The page title is 'Brokers' and the version is '83b5a60 v0.7.2'. The left sidebar shows 'Dashboard', 'local-kafka', 'Brokers', 'Topics', and 'Consumers'. The main content area has a 'Uptime' section with metrics: Broker Count (2), Active Controller (2), Version (1.0-UNKNOWN), Online (0 of 0), URP (0), In Sync Replicas (0 of 0), and Out Of Sync Replicas (0). Below this is a table with columns: Broker ID, Disk usage, Partitions skew, Leaders, Leader skew, Online partitions, Port, and Host. The table lists two brokers: Broker 1 (N/A disk usage, - partitions skew, - leaders, - leader skew, - online partitions, port 9092, localhost) and Broker 2 (N/A disk usage, - partitions skew, - leaders, - leader skew, - online partitions, port 9094, localhost).

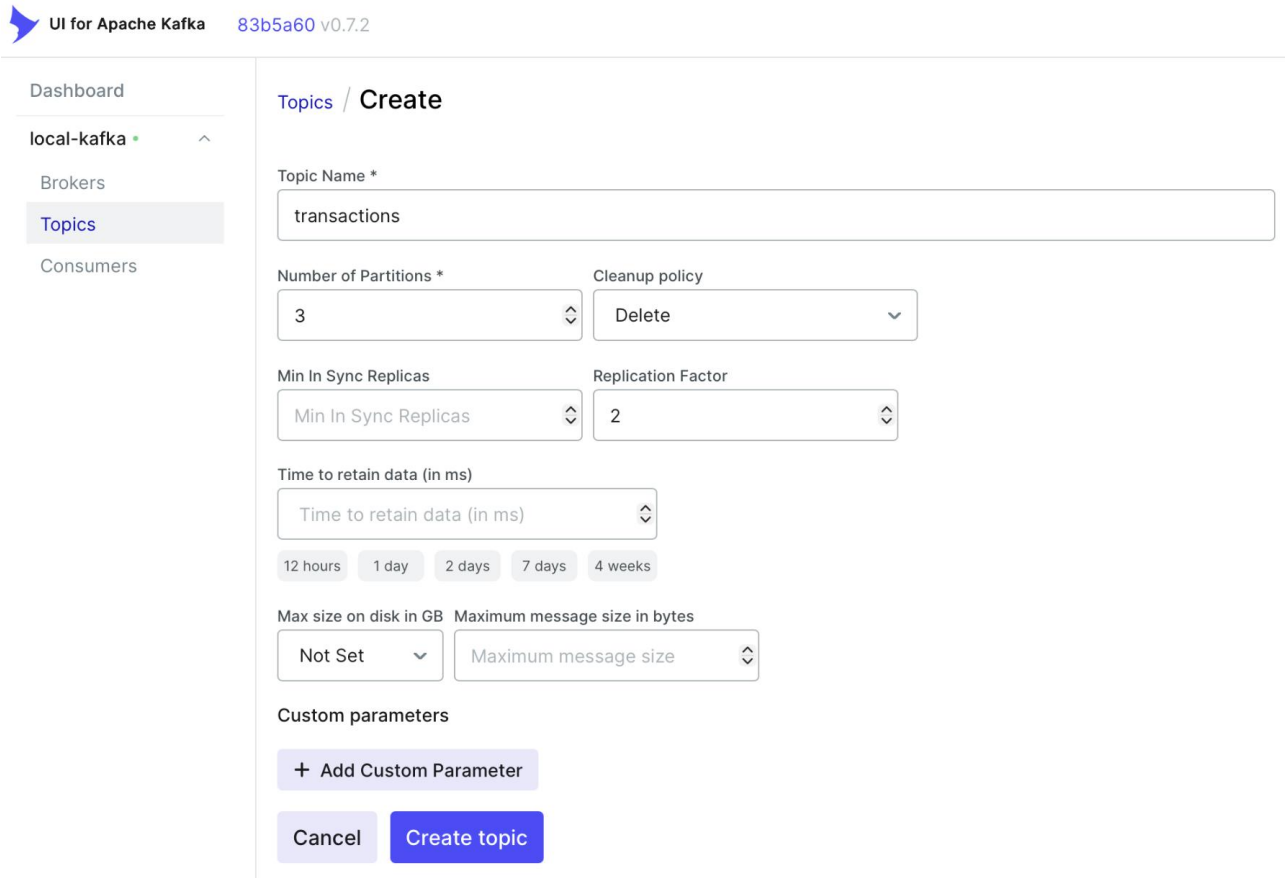
Creating the transactions Topic

I also manually created the topic in my Kafka User Interface while using the following configuration settings:

Number of Partitions: 3

Replication Factor: 2

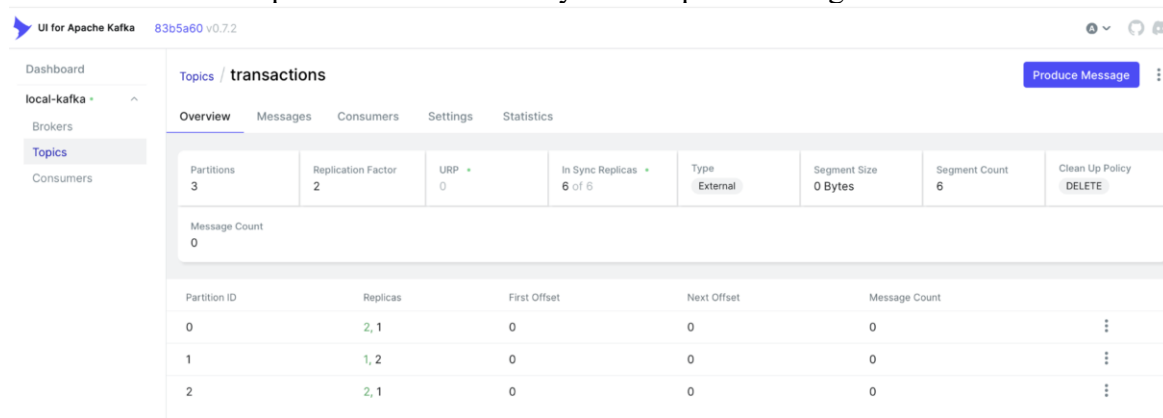
SS 5: Created a transaction topic with a replication factor of two and three partitions



The screenshot shows the 'Topics / Create' page in the Kafka UI. The page title is 'Topics / Create' and the version is '83b5a60 v0.7.2'. The left sidebar shows 'Dashboard', 'local-kafka', 'Brokers', 'Topics', and 'Consumers'. The main content area has a 'Topic Name' field with the value 'transactions'. Below this are two rows of configuration settings: 'Number of Partitions' (3) and 'Cleanup policy' (Delete), 'Min In Sync Replicas' (Min In Sync Replicas) and 'Replication Factor' (2). Below these are two rows of configuration settings: 'Time to retain data (in ms)' (Time to retain data (in ms)) and 'Max size on disk in GB' (Not Set), 'Maximum message size in bytes' (Maximum message size). At the bottom are two buttons: 'Cancel' and 'Create topic'.

Also, I wanted to check that the topic covered requirements. Topic overview showed:
Partitions: 3
Replication Factor: 2
Message Count: 0

SS 6: transactions topic created successfully with required configuration



The screenshot shows the Apache Kafka UI for the 'transactions' topic. The left sidebar has a 'Topics' menu item selected. The main area shows the 'Overview' tab for the 'transactions' topic. It displays various configuration parameters: Partitions (3), Replication Factor (2), URP (0), In Sync Replicas (6 of 6), Type (External), Segment Size (0 Bytes), Segment Count (6), and Clean Up Policy (DELETE). Below these, a table shows the message count for each partition (0 for all three). At the bottom, a detailed table lists the partitions with their IDs, replicas, first and next offsets, and message counts.

Partitions	Replication Factor	URP	In Sync Replicas	Type	Segment Size	Segment Count	Clean Up Policy
3	2	0	6 of 6	External	0 Bytes	6	DELETE

Message Count
0

Partition ID	Replicas	First Offset	Next Offset	Message Count
0	2, 1	0	0	0
1	1, 2	0	0	0
2	2, 1	0	0	0

3. Task 3

Producer Implementation

I The Python program I used to create a producer used **bank_transactions.csv** as its source of transaction records. The contents of the CSV file include the following fields for each row: **transaction_id**, **user_id**, **amount**, **country**, **merchant**, **timestamp**. Each row from the CSV file is read as a dictionary, and then each dictionary is converted to JSON format and sent to Kafka.

SS 7: Kafka producer code

```
UW PICO 5.09 File: producer.py

import csv
import json
import time
from kafka import KafkaProducer

TOPIC_NAME = "transactions"
CSV_FILE = "bank_transactions.csv"
BOOTSTRAP_SERVERS = ["localhost:9092", "localhost:9094"]

producer = KafkaProducer(
    bootstrap_servers=BOOTSTRAP_SERVERS,
    key_serializer=lambda key: str(key).encode("utf-8"),
    value_serializer=lambda value: json.dumps(value).encode("utf-8")
)

print("Kafka Producer started...")
print(f"Reading data from: {CSV_FILE}")
print(f"Sending messages to topic: {TOPIC_NAME}")
print("Each message will be sent with a 10-second delay.")
print("-" * 70)

with open(CSV_FILE, mode="r", encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for row in reader:
        user_id = row["user_id"]

        producer.send(
            TOPIC_NAME,
            key=user_id,
            value=row
        )

        producer.flush()

    print()
```

Following Kafka topic was configured by the producer for message sending:
transactions
The Kafka bootstrap servers were set to these values:
localhost:9092
localhost:9094
The producer used user_id as the Kafka message key:

```
"user_id = row["user_id"]
```

```
producer.send(  
    TOPIC_NAME,  
    key=user_id,  
    value=row  
) "
```

In Kafka, the message key for each transaction message is the user_id value.

Sending Messages with Delay

To simulate a real-time transaction stream, the producer will send each transaction with a 10-second delay between messages: **time.sleep(10)**; this will allow the Kafka UI to see them appearing gradually, as the producer is still running.

Producer Execution

The producer was run in the terminal using this command: ***"python producer.py"***

After starting, the producer reads transactions from the CSV file and sends them to the transactions topic. The terminal output will include the following fields for each sent message: transaction key, transaction ID, amount and country.

SS 8: Kafka producer running and sending transactions

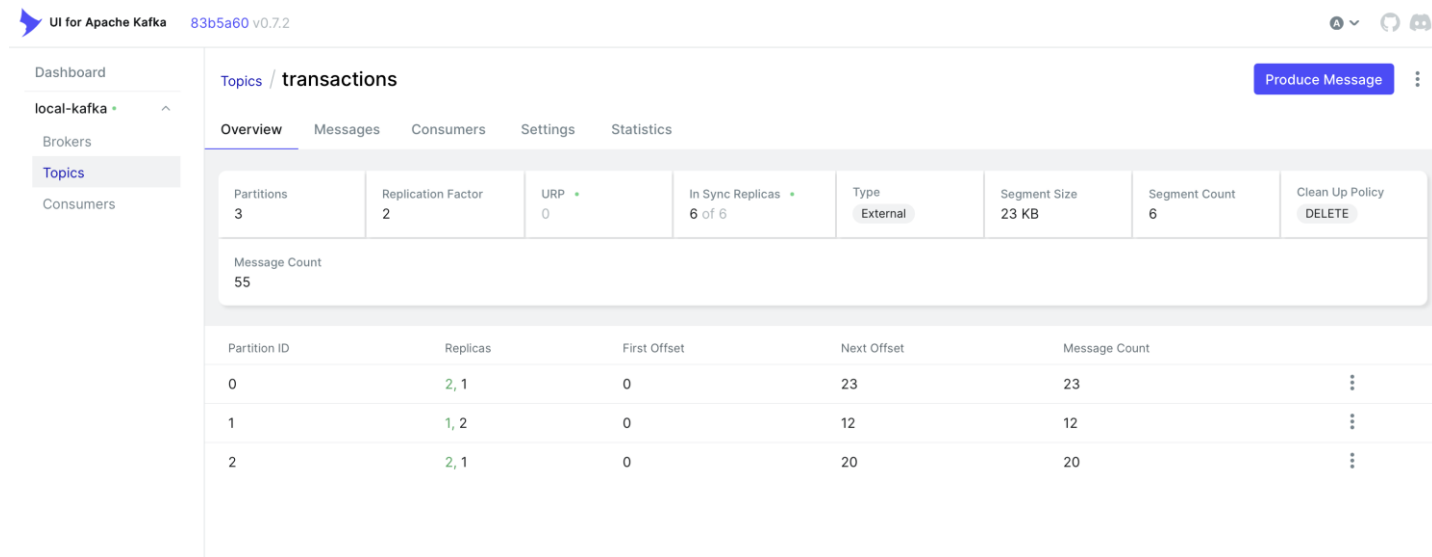
```
(base) metanet@metanets-MacBook-Air kafka-assignment % nano producer.py
(base) metanet@Metanets-MacBook-Air kafka-assignment % python producer.py
Kafka Producer started...
Reading data from: bank_transactions.csv
Sending messages to topic: transactions
Each message will be sent with a 10-second delay.

-----
Sent | key=U001 | transaction=T001 | amount=25 | country=AZ
Sent | key=U002 | transaction=T002 | amount=950 | country=AZ
Sent | key=U001 | transaction=T003 | amount=1200 | country=TR
Sent | key=U003 | transaction=T004 | amount=15 | country=AZ
Sent | key=U004 | transaction=T005 | amount=45 | country=AZ
Sent | key=U005 | transaction=T006 | amount=2200 | country=RU
Sent | key=U002 | transaction=T007 | amount=85 | country=AZ
Sent | key=U006 | transaction=T008 | amount=760 | country=AZ
Sent | key=U007 | transaction=T009 | amount=1350 | country=GE
Sent | key=U003 | transaction=T010 | amount=55 | country=AZ
Sent | key=U008 | transaction=T011 | amount=15 | country=AZ
Sent | key=U009 | transaction=T012 | amount=5000 | country=US
Sent | key=U010 | transaction=T013 | amount=120 | country=AZ
Sent | key=U011 | transaction=T014 | amount=70 | country=AZ
Sent | key=U001 | transaction=T015 | amount=980 | country=AZ
Sent | key=U012 | transaction=T016 | amount=40 | country=AZ
Sent | key=U013 | transaction=T017 | amount=1750 | country=DE
Sent | key=U014 | transaction=T018 | amount=22 | country=AZ
Sent | key=U015 | transaction=T019 | amount=305 | country=AZ
Sent | key=U016 | transaction=T020 | amount=2500 | country=TR
Sent | key=U017 | transaction=T021 | amount=18 | country=AZ
Sent | key=U018 | transaction=T022 | amount=90 | country=AZ
Sent | key=U019 | transaction=T023 | amount=1120 | country=UK
Sent | key=U020 | transaction=T024 | amount=35 | country=AZ
Sent | key=U021 | transaction=T025 | amount=15 | country=AZ
Sent | key=U022 | transaction=T026 | amount=1450 | country=FR
Sent | key=U023 | transaction=T027 | amount=48 | country=AZ
```

Messages Verification in Kafka UI

After running the producer, I verified that it had inserted messages into the transactions topic.

SS 9: Messages visible in Kafka UI



Partition ID	Replicas	First Offset	Next Offset	Message Count
0	2, 1	0	23	23
1	1, 2	0	12	12
2	2, 1	0	20	20

Kafka UI showed that it had inserted messages into the transactions topic. Looking at the Kafka UI, the transaction topic also reports that the total message count of the transaction topic increased as follows:

Total Message Count: 31

This indicates that transaction messages have been successfully produced and saved to the transaction topic.

4. Task 4

Consumer Implementation

I implemented the consumer using Python.

The consumer was configured to read messages from the Kafka topic: transactions

The consumer group was set to:

fraud-detection-group

The Kafka bootstrap servers were configured as:

localhost:9092

localhost:9094

SS 10: Kafka consumer code for fraud detection

```
import json
from kafka import KafkaConsumer

TOPIC_NAME = "transactions"
BOOTSTRAP_SERVERS = ["localhost:9092", "localhost:9094"]
GROUP_ID = "fraud-detection-group"

consumer = KafkaConsumer(
    TOPIC_NAME,
    bootstrap_servers=BOOTSTRAP_SERVERS,
    group_id=GROUP_ID,
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    key_deserializer=lambda key: key.decode("utf-8") if key else None,
    value_deserializer=lambda value: json.loads(value.decode("utf-8"))
)

print("Kafka Consumer started...")
print(f"Reading messages from topic: {TOPIC_NAME}")
print(f"Consumer group: {GROUP_ID}")
print("Fraud rules: amount > 1000 OR country != AZ")
print("-" * 90)

for message in consumer:
    transaction = message.value

    transaction_id = transaction["transaction_id"]
    amount = float(transaction["amount"])
    country = transaction["country"]

    if amount > 1000 or country != "AZ":
        status = "FRAUD"
    else:
        status = "NORMAL"

    print(
        f"Key={message.key} | "
        f"Partition={message.partition} | "
        f"Offset={message.offset} | "
        f"Transaction={transaction_id} | "
        f"Amount={amount} | "
        f"Country={country} | "
        f"Status={status}"
    )
```

Fraud Detection Rules

After consuming the transactions from Kafka, I processed the fraud detection logic for each message. A transaction will be evaluated as fraudulent if it meets the fraud detection rules (the outcome is "fraud", else it will be identified as "normal"). To determine if a transaction meets the fraud detection rules, there are two conditions:

Transaction amount greater than 1000

Transaction was not sent from AZ

The fraud detection logic embedded in the consumer (see code snippet) evaluated each transaction individually. Example code (past the indents there's more code below):

```
if amount > 1000 or country != 'AZ':
    transaction_status = 'FRAUD'
else:
    transaction_status = 'NORMAL'
```

This matches the requirements specified in the assignment.

Consumer Execution

To run the consumer, I executed the following at the command prompt:

```
python consumer.py
```


Once executed, the consumer processed messages from the transactions topic using the fraud-detection consumer group. See below for consumer output that lists both normal and fraudulent transaction messages depicting both key, partition, and offset information

SS 11: Consumer output showing normal and fraud transactions with key, partition, and offset

```
(base) metanet@Metanets-MacBook-Air kafka-assignment % nano producer.py
(base) metanet@Metanets-MacBook-Air kafka-assignment % nano consumer.py
(base) metanet@Metanets-MacBook-Air kafka-assignment % python consumer.py
Kafka Consumer started...
Reading messages from topic: transactions
Consumer group: fraud-detection-group
Fraud rules: amount > 1000 OR country != AZ
-----
Key=U001 | Partition=0 | Offset=0 | Transaction=T001 | Amount=25.0 | Country=AZ | Status=NORMAL
Key=U002 | Partition=0 | Offset=1 | Transaction=T002 | Amount=950.0 | Country=AZ | Status=NORMAL
Key=U001 | Partition=0 | Offset=2 | Transaction=T003 | Amount=1200.0 | Country=TR | Status=FRAUD
Key=U005 | Partition=0 | Offset=3 | Transaction=T006 | Amount=2200.0 | Country=RU | Status=FRAUD
Key=U002 | Partition=0 | Offset=4 | Transaction=T007 | Amount=85.0 | Country=AZ | Status=NORMAL
Key=U006 | Partition=0 | Offset=5 | Transaction=T008 | Amount=760.0 | Country=AZ | Status=NORMAL
Key=U009 | Partition=0 | Offset=6 | Transaction=T012 | Amount=5000.0 | Country=US | Status=FRAUD
Key=U010 | Partition=0 | Offset=7 | Transaction=T013 | Amount=120.0 | Country=AZ | Status=NORMAL
```

For each consumed message, the consumer printed:

Kafka message key

Partition number

Offset

Transaction ID

Amount

Country

Fraud detection status

5. Bonus assignment

SS 12: Bonus parallel consumer code

```
import json
import sys
from kafka import KafkaConsumer

TOPIC_NAME = "bonus-transactions"
BOOTSTRAP_SERVERS = ["localhost:9092", "localhost:9094"]
GROUP_ID = "bonus-consumer-group"

if len(sys.argv) > 1:
    CONSUMER_NAME = sys.argv[1]
else:
    CONSUMER_NAME = "Consumer-Unknown"

consumer = KafkaConsumer(
    TOPIC_NAME,
    bootstrap_servers=BOOTSTRAP_SERVERS,
    group_id=GROUP_ID,
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    key_deserializer=lambda key: key.decode("utf-8") if key else None,
    value_deserializer=lambda value: json.loads(value.decode("utf-8"))
)

print(f"{CONSUMER_NAME} started...")
print(f"Topic: {TOPIC_NAME}")
print(f"Consumer group: {GROUP_ID}")
print("Waiting for messages...")
print("-" * 90)

for message in consumer:
    transaction = message.value

    print(
        f"{CONSUMER_NAME} | "
        f"Partition={message.partition} | "
        f"Offset={message.offset} | "
        f"Transaction={transaction['transaction_id']}"
    )
```

Bonus Topic and Setup

In order to clearly show the concept of parallel processing (with separate and concurrent execution of multiple threads on multiple CPUs), I created a new Kafka topic (as outlined in Chapter 1) with the following characteristics:

Topic Name: bonus-video-transactions

Number of Partitions: 3

Replication Factor: 2

By having three (3) partitions, Kafka was able to distribute the partitions amongst two (2) consumers who belong to the same consumer group. The consumer group that I used for the bonus task: bonus-video-consumer-group

Parallel Consumer Implementation

I utilized Python for my Consumer implementation. The same Consumer code allows for two separate Consumers to run by simply passing in a unique Consumer name. In both cases, we are within the same Kafka Consumer group of bonus-video-consumer-group

Figure 12: Bonus parallel consumer code

A screenshot of a terminal window titled 'UW PICO 5.09' with a 'File' menu. The terminal displays Python code for a Kafka consumer. The code imports 'json' and 'sys' from the standard library, and 'KafkaConsumer' from the 'kafka' module. It defines constants for 'TOPIC_NAME' ('bonus-video-transactions'), 'BOOTSTRAP_SERVERS' (['localhost:9092', 'localhost:9094']), and 'GROUP_ID' ('bonus-video-consumer-group'). It checks for command-line arguments to set 'CONSUMER_NAME', defaulting to 'Consumer-Unknown'. A 'KafkaConsumer' object is instantiated with these parameters, including 'auto_offset_reset="earliest"', 'enable_auto_commit=True', and custom deserializers for keys and values. The code prints startup information and then enters a loop to consume messages, printing details for each transaction. At the bottom, there are keyboard shortcuts: ^G Get Help, ^X Exit, ^O WriteOut, ^J Justify, ^R Read File, and ^W Where is.

```
UW PICO 5.09 File
import json
import sys
from kafka import KafkaConsumer

TOPIC_NAME = "bonus-video-transactions"
BOOTSTRAP_SERVERS = ["localhost:9092", "localhost:9094"]
GROUP_ID = "bonus-video-consumer-group"

if len(sys.argv) > 1:
    CONSUMER_NAME = sys.argv[1]
else:
    CONSUMER_NAME = "Consumer-Unknown"

consumer = KafkaConsumer(
    TOPIC_NAME,
    bootstrap_servers=BOOTSTRAP_SERVERS,
    group_id=GROUP_ID,
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    key_deserializer=lambda key: key.decode("utf-8") if key else None,
    value_deserializer=lambda value: json.loads(value.decode("utf-8"))
)

print(f"{CONSUMER_NAME} started...")
print(f"Topic: {TOPIC_NAME}")
print(f"Consumer group: {GROUP_ID}")
print("Waiting for messages...")
print("-" * 90)

for message in consumer:
    transaction = message.value

    print(
        f"{CONSUMER_NAME} | "
        f"Partition={message.partition} | "
        f"Offset={message.offset} | "
        f"Transaction={transaction['transaction_id']}"
    )

^G Get Help    ^O WriteOut    ^R Read File
^X Exit        ^J Justify     ^W Where is
```

The consumer printed the consumer name, partition number, offset, and transaction ID for each consumed message:

Consumer-1 | Partition=2 | Offset=0 | Transaction=T004

Consumer-2 | Partition=0 | Offset=0 | Transaction=T001

Bonus Producer Execution

I used separate producer to send messages to the bonus-video-transactions topic. The producer read transaction records from the bank_transactions.csv file and sent them to Kafka.

SS 13: Bonus producer sending messages to bonus-transactions topic

```
Last login: Mon May 11 21:51:30 on ttys007
(base) metanet@Metanets-MacBook-Air ~ % cd ~/kafka-assignment
[python bonus_producer.py
Bonus Producer started...
Sending messages to topic: bonus-transactions
-----
Sent | key=U001 | transaction=T001 | amount=25 | country=AZ
Sent | key=U002 | transaction=T002 | amount=950 | country=AZ
Sent | key=U001 | transaction=T003 | amount=1200 | country=TR
Sent | key=U003 | transaction=T004 | amount=15 | country=AZ
Sent | key=U004 | transaction=T005 | amount=45 | country=AZ
Sent | key=U005 | transaction=T006 | amount=2200 | country=RU
Sent | key=U002 | transaction=T007 | amount=85 | country=AZ
Sent | key=U006 | transaction=T008 | amount=760 | country=AZ
Sent | key=U007 | transaction=T009 | amount=1350 | country=GE
Sent | key=U003 | transaction=T010 | amount=55 | country=AZ
Sent | key=U008 | transaction=T011 | amount=15 | country=AZ
Sent | key=U009 | transaction=T012 | amount=5000 | country=US
Sent | key=U010 | transaction=T013 | amount=120 | country=AZ
Sent | key=U011 | transaction=T014 | amount=70 | country=AZ
Sent | key=U001 | transaction=T015 | amount=980 | country=AZ
Sent | key=U012 | transaction=T016 | amount=40 | country=AZ
Sent | key=U013 | transaction=T017 | amount=1750 | country=DE
Sent | key=U014 | transaction=T018 | amount=22 | country=AZ
Sent | key=U015 | transaction=T019 | amount=305 | country=AZ
Sent | key=U016 | transaction=T020 | amount=2500 | country=TR
Sent | key=U017 | transaction=T021 | amount=18 | country=AZ
Sent | key=U018 | transaction=T022 | amount=90 | country=AZ
Sent | key=U019 | transaction=T023 | amount=1120 | country=UK
Sent | key=U020 | transaction=T024 | amount=35 | country=AZ
Sent | key=U021 | transaction=T025 | amount=15 | country=AZ
Sent | key=U022 | transaction=T026 | amount=1450 | country=FR
Sent | key=U023 | transaction=T027 | amount=48 | country=AZ
Sent | key=U024 | transaction=T028 | amount=75 | country=AZ
Sent | key=U025 | transaction=T029 | amount=600 | country=AZ
Sent | key=U026 | transaction=T030 | amount=3200 | country=CA
Sent | key=U027 | transaction=T031 | amount=44 | country=AZ
Sent | key=U028 | transaction=T032 | amount=89 | country=AZ
Sent | key=U029 | transaction=T033 | amount=1550 | country=TR
Sent | key=U030 | transaction=T034 | amount=28 | country=AZ
Sent | key=U031 | transaction=T035 | amount=65 | country=AZ
Sent | key=U032 | transaction=T036 | amount=2100 | country=RU
Sent | key=U033 | transaction=T037 | amount=15 | country=AZ
Sent | key=U034 | transaction=T038 | amount=730 | country=AZ
```

Demonstrating Parallel Processing

By running the producer after starting both consumers I was able to allow Kafka to assign partitions to each of the consumers in that consumer group.

SS 14: Two consumers running simultaneously and processing messages in parallel

```
metanet - python bonus_consumer.py Consumer-1 - 99x53
Last login: Mon May 11 18:09:56 on ttys006
(base) metanet@Metanets-MacBook-Air ~ % cd ~/kafka-assignment
python bonus_consumer.py Consumer-1
Consumer-1 started...
Topic: bonus-transactions
Consumer group: bonus-consumer-group
Waiting for messages...

Consumer-1 | Partition=2 | Offset=0 | Transaction=T004
Consumer-1 | Partition=2 | Offset=1 | Transaction=T009
Consumer-1 | Partition=2 | Offset=2 | Transaction=T010
Consumer-1 | Partition=2 | Offset=3 | Transaction=T014
Consumer-1 | Partition=2 | Offset=4 | Transaction=T021
Consumer-1 | Partition=2 | Offset=5 | Transaction=T023
Consumer-1 | Partition=2 | Offset=6 | Transaction=T025
Consumer-1 | Partition=2 | Offset=7 | Transaction=T026
Consumer-1 | Partition=2 | Offset=8 | Transaction=T027
Consumer-1 | Partition=2 | Offset=9 | Transaction=T028
Consumer-1 | Partition=2 | Offset=10 | Transaction=T031
Consumer-1 | Partition=2 | Offset=11 | Transaction=T034
Consumer-1 | Partition=2 | Offset=12 | Transaction=T035
Consumer-1 | Partition=2 | Offset=13 | Transaction=T038
Consumer-1 | Partition=2 | Offset=14 | Transaction=T040
Consumer-1 | Partition=2 | Offset=15 | Transaction=T042
Consumer-1 | Partition=2 | Offset=16 | Transaction=T044
Consumer-1 | Partition=2 | Offset=17 | Transaction=T047
Consumer-1 | Partition=2 | Offset=18 | Transaction=T053
Consumer-1 | Partition=2 | Offset=19 | Transaction=T054
Consumer-1 | Partition=2 | Offset=20 | Transaction=T061

metanet - python bonus_consumer.py Consumer-2 - 99x53
Last login: Mon May 11 21:50:34 on ttys006
(base) metanet@Metanets-MacBook-Air ~ % cd ~/kafka-assignment
python bonus_consumer.py Consumer-2
Consumer-2 started...
Topic: bonus-transactions
Consumer group: bonus-consumer-group
Waiting for messages...

Consumer-2 | Partition=0 | Offset=0 | Transaction=T001
Consumer-2 | Partition=0 | Offset=1 | Transaction=T002
Consumer-2 | Partition=0 | Offset=2 | Transaction=T003
Consumer-2 | Partition=1 | Offset=0 | Transaction=T005
Consumer-2 | Partition=0 | Offset=3 | Transaction=T006
Consumer-2 | Partition=0 | Offset=4 | Transaction=T007
Consumer-2 | Partition=0 | Offset=5 | Transaction=T008
Consumer-2 | Partition=1 | Offset=1 | Transaction=T011
Consumer-2 | Partition=0 | Offset=6 | Transaction=T012
Consumer-2 | Partition=0 | Offset=7 | Transaction=T013
Consumer-2 | Partition=0 | Offset=8 | Transaction=T015
Consumer-2 | Partition=1 | Offset=2 | Transaction=T016
Consumer-2 | Partition=1 | Offset=3 | Transaction=T017
Consumer-2 | Partition=1 | Offset=4 | Transaction=T018
Consumer-2 | Partition=0 | Offset=9 | Transaction=T019
Consumer-2 | Partition=1 | Offset=5 | Transaction=T020
Consumer-2 | Partition=0 | Offset=10 | Transaction=T022
Consumer-2 | Partition=1 | Offset=6 | Transaction=T024
Consumer-2 | Partition=0 | Offset=11 | Transaction=T029
Consumer-2 | Partition=0 | Offset=12 | Transaction=T030
Consumer-2 | Partition=0 | Offset=13 | Transaction=T032
Consumer-2 | Partition=0 | Offset=14 | Transaction=T033
Consumer-2 | Partition=1 | Offset=7 | Transaction=T036
Consumer-2 | Partition=0 | Offset=15 | Transaction=T037
Consumer-2 | Partition=1 | Offset=8 | Transaction=T039
Consumer-2 | Partition=1 | Offset=9 | Transaction=T041
Consumer-2 | Partition=0 | Offset=16 | Transaction=T043
Consumer-2 | Partition=0 | Offset=17 | Transaction=T045
Consumer-2 | Partition=0 | Offset=18 | Transaction=T046
Consumer-2 | Partition=0 | Offset=19 | Transaction=T048
Consumer-2 | Partition=1 | Offset=10 | Transaction=T049
Consumer-2 | Partition=0 | Offset=20 | Transaction=T050
Consumer-2 | Partition=0 | Offset=21 | Transaction=T051
Consumer-2 | Partition=0 | Offset=22 | Transaction=T052 Consumer-2 | Partition=1 | Offset=11 | Transaction=T055
Consumer-2 | Partition=0 | Offset=23 | Transaction=T056
Consumer-2 | Partition=0 | Offset=24 | Transaction=T057
Consumer-2 | Partition=0 | Offset=25 | Transaction=T058
Consumer-2 | Partition=1 | Offset=12 | Transaction=T059
Consumer-2 | Partition=1 | Offset=13 | Transaction=T060
Consumer-2 | Partition=0 | Offset=26 | Transaction=T062
```

This shows that Kafka was able to dispatch the partitions as well as all the messages across the two consumers. In this case, Consumer-1 was processing messages out of one partition, and Consumer-2 was processing messages from the remaining partitions.

6. YouTube video

Link to YouTube video: https://youtu.be/o3tC_8TqSLE?si=s0mtsmpHWthowsPH